

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

If allocated, display the block number, block size, and internal

Completed

Select Language : C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int m, n, i, j;
5
6     printf("Enter number of memory blocks:\n");
7     scanf("%d", &m);
8
9     int block[m], original[m];
10
11     printf("Enter size of each block:\n");
12     for(i = 0; i < m; i++) {
13         scanf("%d", &block[i]);
14         original[i] = block[i]; // store original
15     }
16
17     printf("Enter number of processes:\n");
18     scanf("%d", &n);
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

[Back to Practice](#)

Best Fit Contiguous Memory Allocation

Completed

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

If allocated, display the block number, block size, and internal

Select Language : C (GCC 9.2.0)

```
20  int process[n];
21
22  printf("Enter size of each process:\n");
23  for(i = 0; i < n; i++) {
24      scanf("%d", &process[i]);
25  }
26
27  for(i = 0; i < n; i++) {
28      int best = -1;
29
30      for(j = 0; j < m; j++) {
31          if(block[j] >= process[i]) {
32              if(best == -1 || block[j] < block[best]) {
33                  best = j;
34              }
35          }
36      }
37
38      if(best != -1) {
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

[Back To Practice](#)

< Q1 >

Save

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

1. The first line contains an integer representing the number of memory blocks.
2. The second line contains the sizes of the memory blocks.
3. The third line contains an integer representing the number of processes.
4. The fourth line contains the sizes of the processes.

Output Format

- For each process, display whether it is allocated or not.

```
37
38     if(best != -1) {
39         int fragment = block[best] - process[i];
40
41         printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",
42             i + 1, process[i], best + 1, original[best], fragment);
43
44         block[best] -= process[i]; // reduce size
45     }
46     else {
47         printf("Process %d of size %d is not allocated\n", i + 1, process[i]);
48     }
49 }
50
51 return 0;
52 }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

> Test Case - 2